When Should Web-RAG Refresh? Adapting Refresh Policies Across Drift Regimes

Anonymous Author(s)

Anonymous Affiliation
anonymous@example.com

Abstract. Web-backed RAG systems cache retrieved pages to cut latency and cost, but cached content grows stale. When should the system pay for a live refresh, and how does the answer change across drift regimes? We study two complementary regimes — FreshQA dev (low drift) and a new TimeVarying benchmark (high drift), separated by Cohen's d=2.15 — and deliver two practical recipes. First, *which* policy is best depends on *how much* the content drifts: blind Always-Refresh is the worst system in low drift (F1 0.173) but the best in high drift (F1 0.705); a channel decomposition attributes the low-drift drop to noise in the refreshed evidence itself. Second, a proxy-trained refresh router (DeBERTa-v3-small) collapses zero-shot across regimes (F1 0.422) but a 5-epoch adaptation on as few as k=4 target queries recovers it to F1 0.685, above every rule-based router; a from-scratch control arm trained with the same protocol and budget on a generic DeBERTa-v3-small init reaches only F1 0.189 at k=4 (and diverges at larger lr), showing that the gain reflects transferred policy structure from FreshQA rather than generic backbone calibration. Leave-one-category-out validation confirms the recovery generalizes. Finally, a corpus-level regime classifier — averaging a cheap drift predictor over ∼20 held-out queries and thresholding — separates the two regimes with no overlap and picks the policy family that matches, or comes within 0.02 of, the best fixed policy, answering the deployment question of *which* policy to ship.
Code: https://anonymous.4open.science/r/wise-rag-refresh-4606

Keywords: Retrieval-augmented generation · Web information retrieval · Temporal query intent · Cache refresh · Domain adaptation · Cost-aware systems

1 Introduction

Web-RAG systems answer questions by retrieving web pages and conditioning an LLM on the evidence. The web is the system's source of truth, and that source is not static: news, prices, and leadership change, and cached pages age. The system's evidence therefore drifts, and the system has no native way to know how much [?]. Recent work estimates how often answers change [?], diagnoses when stale retrieval context hurts generation [?], and benchmarks agents on evolving knowledge [?]. Yet a cost-aware refresh policy, deciding per query

whether to pay for a live page, remains unstudied at the system level: temporal-IR work optimizes ranking, not downstream generation; cache-refresh heuristics evaluate freshness, not answer quality; and adaptive-retrieval methods [**?**,**?**] do not condition on corpus age.

This paper asks: *when should a web-backed RAG system refresh cached evidence, and how does the answer change across drift regimes?*

- **RQ1.** How does staleness degrade RAG answer quality, and does the degradation depend on query type?
- **RQ2.** Does selective refresh beat blind refresh on the cost/accuracy frontier, and does the answer depend on the drift regime? *And if the answer does depend on drift, how does a real system pick which policy to ship?*
- **RQ3.** Do trained refresh routers transfer across corpora, and can a small adaptation step recalibrate them?

We answer these on FreshQA (low drift, real Wayback snapshots [**?**]) and a curated TimeVarying benchmark (high drift, n=60 across 5 categories), separated by Cohen's d=2.15. We propose two practical recipes that come out of this study. First, a **transfer-and-adapt recipe**: although proxy-trained routers collapse zero-shot across regimes (F1 0.188 $\rightarrow$ 0.422), a 5-epoch adaptation on $k \geq 4$ target queries recovers them to F1 0.685, with the gain saturating at k=4. Second, a **corpus-level regime classifier**: averaging a stale-side drift predictor over a held-out batch of $\sim$20 queries and thresholding separates the two drift regimes with no overlap and picks the policy family that matches (low-drift) or comes within 0.02 of (high-drift) the best fixed policy — answering, at near-zero per-query cost, the deployment question of *which* policy to ship. The two recipes cover the deployment question from both ends: the regime classifier decides *which family* of policy to run, and the adapted router is the high-drift policy that family should ship.

Contributions.

- **A two-corpus diagnostic framework** that separates a low-drift regime (FreshQA, real Wayback snapshots) from a high-drift regime (TimeVarying, n=60 across 5 categories), with a measured drift gap of Cohen's d=2.15, and a controlled decomposition (§4.2) of the counter-intuitive low-drift drop.
- **A regime-dependent refresh principle**: the value of blind vs. selective refresh is governed by corpus drift regime; selective refresh recovers most of the high-drift benefit of full refresh at much lower cost.
- **A transfer-and-adaptation recipe** for proxy-trained refresh routers: the same router that fails zero-shot across corpora recovers with as few as k=4 target labels.
- **A corpus-level regime classifier** that, given only stale-side drift estimates, selects which refresh policy family a real system should deploy — turning the drift-regime observation into a deployment answer.

– **An empirical scope result on drift prediction**: per-query drift is predictable from stale-side signals but does not translate into better *per-query* refresh policy selection at our data scale; the useful level of granularity for the predictor is the corpus, not the query.

2 Related Work

Temporal retrieval and web freshness. Time-aware ranking and recency-sensitive retrieval model *when* documents are relevant [?,?]. FreshQA [?] provides time-sensitive QA data but does not study refresh *policies*. Recent work estimates recency requirements [?], diagnoses when stale context hurts generation [?], and benchmarks agents on evolving knowledge [?]. These works do not study cost-aware refresh or cross-regime transfer of refresh policies.

Adaptive retrieval and cross-corpus transfer. Self-RAG [?], CRAG [?], RAPTOR [?], FLARE [?], and SeaKR [?] decide *whether or when* to retrieve from a static index, but treat the corpus as static and do not condition on corpus *age*. BEIR [?] showed that dense retrievers transfer unevenly across corpora; we apply this to the refresh-policy setting. Calibrated retrieval-budget allocation [?] is adjacent but does not address live-refresh cost. Our work isolates when selective live-refresh against a stale cache helps, and when proxy-trained routers fail to transfer across drift regimes.

Cache refresh and our position relative to it. Classical web-cache refresh [?] and time-to-live (TTL) heuristics optimize *freshness of the cache* as the objective; the trade-off is between staleness and refresh frequency. RAG refresh is a different problem: the unit of work is the *answer quality of a downstream LLM*, and the trade-off is between that quality and refresh cost (latency, dollars). The cache-refresh literature has no notion of an LLM-conditioned reader, and an LLM-conditioned reader is sensitive to *which* document changes, not just *whether* something has changed — a refreshed Wikipedia paragraph can introduce a stylistic or topical shift that hurts answer quality even when the gold answer is unchanged. Our regime-dependent result is therefore not derivable from TTL heuristics: in our low-drift setting, blind refresh *worsens* answer quality even though it strictly improves freshness, the opposite of what a TTL policy would predict.

Transfer learning and low-shot adaptation. Transfer-and-adapt recipes are well-studied in NLP [?,?] and have been used to recalibrate ranking models across corpora [?]. Our adaptation step is in this lineage but specialized: the target supervision is not a label but a scalar *refresh utility*, and the model is a small regression head on top of a frozen-source representation. The sample-flat saturation we observe (k=4 already covers most of the gain) is consistent with the low-dimensional calibration view of [?].

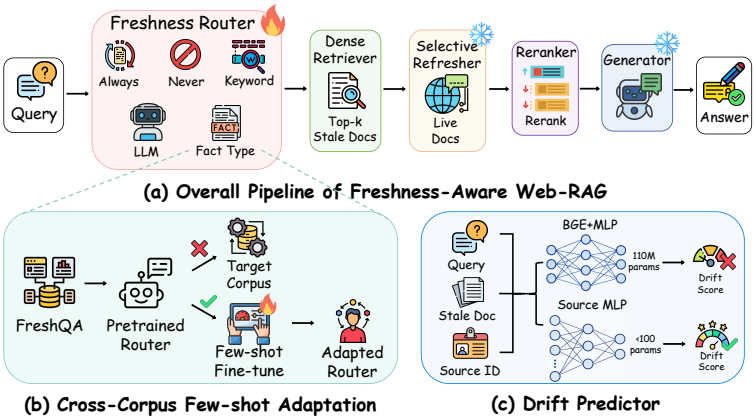


Fig. 1. Overall framework of the freshness-aware Web-RAG pipeline.

3 Method and Setup

Overview. We study *adaptive refresh control* for Web-RAG under heterogeneous corpus drift regimes. The goal is to decide, for each incoming query, whether the retrieval index should be refreshed before generation, so as to maximize answer quality while minimizing unnecessary refresh operations. Compared with a static refresh schedule, our method explicitly models drift signals, predicts the utility of refreshing, and adapts this predictor to a new corpus using only a small number of labeled examples (empirically saturating at $k = 4$).

3.1 Problem formulation

We formalize adaptive refresh as a sequential decision problem. A stream of queries q_t arrives against a corpus state C_t and a retrieval index I_t currently used by the system; a refresh action updates I_t to match C_t, whereas reuse keeps the previous index. For each query the policy chooses an action $a_t \in \{0, 1\}$ (a_t=1 denotes `Refresh`), and $F_1(q_t, a_t; C_t, I_t)$ denotes the answer F1 obtained by the downstream Web-RAG pipeline under that action. Refreshing also incurs a real-world cost — a live fetch of the page — that is *external* to answer quality: it is paid in latency and fetch traffic, not in F1. We count one unit of this cost per refresh, $c(a_t) = a_t$, so the average refresh rate of a policy π is $\mathrm{RR}(\pi) = \frac{1}{T}\sum_t \mathbb{E}_{a_t\sim\pi}[a_t]$, and the trade-off weight λ in the objective below expresses how many F1 points one refresh is worth to the operator. (We keep the c notation only to write the objective cleanly; in our experiments c is exactly the refresh indicator, and we report F1 and refresh rate separately rather than collapsing them into a single scalar.) Here $x_t \in \mathbb{R}^d$ is the observable feature vector extracted before answering q_t; in our implementation $d = 6$, and the six scalar features summarize query novelty, retrieval instability, temporal staleness, and corpus-update statistics.

The objective is to learn a routing policy π that maximizes expected utility under an explicit quality–cost trade-off:

$$\max_{\pi \in \Pi} \mathcal{J}(\pi) = \frac{1}{T} \sum_{t=1}^{T} \mathbb{E}_{a_t \sim \pi(\cdot|x_t)} \Big[F_1(q_t, a_t; C_t, I_t) - \lambda c(a_t) \Big], \tag{1}$$

where $\lambda \geq 0$ controls the penalty on refresh frequency (equivalently, F1 maximization subject to an average refresh budget ρ, with λ the associated Lagrange multiplier).

For each query, define the *refresh gain*

$$\Delta_t = F_1(q_t, 1; C_t, I_t) - F_1(q_t, 0; C_t, I_t). \tag{2}$$

Then the optimal Bayes action under objective (1) is

$$a_t^\star = \mathbf{1}\{\Delta_t > \lambda\}, \tag{3}$$

which shows that adaptive refresh reduces to estimating whether the expected benefit of refreshing exceeds its cost threshold. Our method therefore learns a predictor $p_{\text{fresh}}(x_t) \approx \Pr(\Delta_t > 0 \mid x_t)$ and converts it to a decision through a calibrated threshold.

3.2 Drift-aware refresh utility prediction

We model latent corpus drift through a scalar drift score $\hat{\delta}_t = f_\theta(x_t)$, where f_θ is a parametric predictor and $x_t \in \mathbb{R}^6$ is the six-dimensional feature vector. Intuitively, $\hat{\delta}_t$ estimates the probability that the current retrieval index is stale *for this query*. We then map $\hat{\delta}_t$ to a refresh utility score

$$p_{\text{fresh}}(x_t) = \sigma\big(g_\phi(\hat{\delta}_t, x_t)\big) = \sigma\big(g_\phi(f_\theta(x_t), x_t)\big), \tag{4}$$

where $\sigma(\cdot)$ is the sigmoid function and g_ϕ is a lightweight affine calibration head, $g_\phi(f_\theta(x_t), x_t) = w^\top x_t + u\hat{\delta}_t + b$.

The router outputs

$$\pi_{\theta,\phi}(x_t) = \mathbf{1}\{p_{\text{fresh}}(x_t) > \tau_\lambda\}, \tag{5}$$

where τ_λ is chosen on validation data to maximize the composite metric $F_1 - \lambda \cdot \text{RR}$. This formulation makes explicit the routing principle behind our empirical findings: the adapted router improves decision quality substantially over non-adaptive baselines while preserving strong query-level alignment with oracle refresh utility (r=0.81).

Why six scalar features suffice. A natural alternative is to encode richer query–corpus context using high-dimensional semantic embeddings (e.g., BGE) followed by an MLP. However, refresh routing is not a semantic matching problem alone; it is a *calibration* problem over a small set of operational statistics. Our

hypothesis is that the refresh gain depends on a low-dimensional latent state, $\Delta_t \approx h(Px_t)$ for a low-rank projection P, so the relevant signal already lies in the compact subspace captured by the six engineered scalars. Learning a stable decision rule directly in the full embedding space from few-shot target labels instead suffers estimation error of order $\mathcal{O}(\sqrt{d_{\text{eff}}/n})$ in the effective model dimension d_{eff} and label count n. When n is very small, as in our setting, reducing d_{eff} matters more than increasing representational capacity, which explains why BGE+MLP underperforms despite higher expressive power.

3.3 Freshness-Aware Web-RAG Pipeline

Our core pipeline is deliberately simple so that differences across policies can be attributed to refresh decisions rather than to unrelated retrieval or generation changes. Given a query, the system routes it through a freshness router, retrieves a candidate set from the stale index, selectively refreshes only documents whose expected utility exceeds the policy threshold, reranks the merged candidate set, and generates the final answer from the top-ranked refreshed context (Fig. 1). The same retriever, reranker, generator, document budget, and evaluation metrics are used for all refresh policies. A refresh action updates I_t to match the latest corpus state C_t; a reuse action keeps the previous index. Importantly, refresh is defined as *same-URL replacement*: if d_i^0 corresponds to URL or document id u_i, then refresh fetches d_i^1 for the same u_i. We do not reissue retrieval against a separate live index, which isolates the effect of evidence freshness from changes in retrieval coverage. Stale retrieval uses BGE-base [?] over $\mathcal{D}^0$; the reranker combines the retrieval score with freshness indicators, $s_{\text{final}} = s_{\text{retrieval}} + \lambda_1 \mathbf{1}_{\text{live}} + \lambda_2 \mathbf{1}_{\text{changed}}$; and the generator is fixed across settings so that changes in F1 reflect the evidence selected by the refresh policy rather than decoding or model-size changes.

3.4 Refresh Policies and Routers

We compare five policy types covering blind, rule-based, zero-shot, proxy-trained, and adapted refresh behavior. (i) *Never Refresh* always sets $a_t = 0$. (ii) *Always Refresh* always sets $a_t = 1$ and refreshes the top-m documents for every query. (iii) *Keyword Router* fires on explicit temporal or update-seeking cues (current, latest, recent, today, now, new, price, population, etc.). (iv) *LLM Router* is a zero-shot semantic baseline that predicts refresh need from the query. (v) *Fact-Type Router* is the main learned router: a DeBERTa-v3-small [?] that predicts a scalar $p_{\text{fresh}}(q)$ as in (4), trained on FreshQA's `fact_type` metadata with the MSE loss in (6). The thresholded output follows (5). Our adaptation step fine-tunes this router on k target-domain queries with the objective of §3.5.

3.5 Supervised adaptation objective

We now formalize the low-shot adaptation stage. For a target corpus $\mathcal{C}^{\text{tar}}$, we collect a small labeled set $\mathcal{D}_k^{\text{tar}} = \{(x_i, \tilde{y}_i)\}_{i=1}^k$, where $\tilde{y}_i \in [0, 1]$ is the supervision

target for refresh utility (derived from the empirical refresh label, i.e., whether refreshing improves answer quality, or from a normalized proxy of that gain). The predictor outputs $p_{\text{fresh}}(x_i)$ as in (4), and adaptation minimizes mean squared error:

$$\mathcal{L}_{\text{adapt}}(\theta, \phi) = \frac{1}{k} \sum_{i=1}^{k} \left(p_{\text{fresh}}(x_i) - \tilde{y}_i\right)^2 . \tag{6}$$

We fine-tune for $E = 5$ epochs with AdamW (learning rate η), which in practice suffices to recalibrate the router to a new corpus regime without overfitting; the adapted policy thresholds the resulting p_{fresh} at τ_λ.

3.6 Label-efficiency hypothesis and saturation at $k = 4$

Our empirical results show that adaptation gains saturate quickly, with most of the improvement achieved by $k = 4$ labeled examples. We state the following hypothesis to explain this phenomenon.

Hypothesis 1 (Low-dimensional corpus calibration). Let the target-corpus-specific routing function belong to a low-dimensional affine family

$$p_{\text{fresh}}^{(c)}(x) = \sigma\left(w_0^\top x + b_0 + U\alpha_c\right), \tag{7}$$

where (w_0, b_0) are shared parameters learned from source corpora, $U \in \mathbb{R}^{(d+1)\times r}$ spans a corpus-specific calibration subspace, and $\alpha_c \in \mathbb{R}^r$ is a corpus-dependent coefficient with small intrinsic dimension r. If r is small (e.g., $r \leq 4$), then only a few target labels are needed to identify α_c up to small estimation error: standard arguments give $\|\hat{\alpha}_c - \alpha_c^\star\|_2^2 = \mathcal{O}(r/k)$, so the induced excess routing loss scales as $\mathcal{O}(r/k)$ and the marginal gain from increasing k becomes small once $k \gtrsim r$. This provides a principled explanation for why performance saturates at k=4 in our experiments. We emphasize that this is a *hypothesis* consistent with the observed data, rather than a formal theorem.

3.7 Cross-corpus transfer and regret

Finally, we formalize cross-corpus transfer through the regret of the adapted router relative to an oracle policy that knows the true refresh gains Δ_t. Let $\pi^\star$ denote the oracle router defined by (3), and let $\hat{\pi}_k$ denote the router adapted with k target labels. With per-query utility $u_t(\pi) = F_1(q_t, \pi(x_t); C_t, I_t) - \lambda c(\pi(x_t))$, the regret on a target corpus is

$$\text{Regret}_k = \frac{1}{T} \sum_{t=1}^{T} \Big(\mathbb{E}[u_t(\pi^\star)] - \mathbb{E}[u_t(\hat{\pi}_k)]\Big). \tag{8}$$

If the adapted predictor's expected calibration error satisfies $\mathbb{E}_{x\sim\mathcal{D}^{\text{tar}}}|p_{\text{fresh}}(x) - p_{\text{fresh}}^\star(x)| \leq \epsilon_k$ with ϵ_k decreasing in k, and the utility is L-Lipschitz in the refresh probability, then $\text{Regret}_k \leq L\,\epsilon_k$. Combining this with the $\mathcal{O}(r/k)$ estimation

error of Hypothesis 1 gives a decreasing regret rate of order $\mathcal{O}(\sqrt{r/k})$. This captures the central intuition of our method: source training provides a strong prior over refresh behavior, while a small number of target labels reduces the residual corpus-specific mismatch.

3.8 Corpora, drift regimes, and metrics

What a drift regime is. Throughout the paper, a *drift regime* is a property of a *corpus*, not of a query: it is the distribution of per-query content drift δ between cached (stale) and current (fresh) versions of the same documents, summarized by the corpus mean $\bar{\delta}$. Concretely, δ_i is the complement of content-word overlap between the stale and fresh text of query i's evidence (δ=0: byte-stable; $\delta \to 1$: fully rewritten). Two corpora are in *different* regimes when their δ distributions are separated — in our case by Cohen's d=2.15. "Low-drift regime" means the cached evidence is largely still correct ($\bar{\delta} \approx 0.03$); "high-drift regime" means it is substantively outdated ($\bar{\delta} \approx 0.17$). The regime matters because the refresh gain Δ_t of (2) inherits the sign of the drift: refreshing stale≈fresh evidence can only perturb, while refreshing stale≠fresh evidence can repair.

We instantiate the framework on two complementary QA corpora spanning a measurable drift gap and one auxiliary probe corpus; full descriptions and construction details are deferred to §4. In brief: FreshQA dev [?] (n=74) is low-drift (mean δ=0.026); our TimeVarying benchmark (n=60 across five categories) is high-drift (mean δ=0.171); and Natural Questions [?] (n=273) serves only as a motivational staleness-cue probe. The drift gap between FreshQA and TimeVarying is Cohen's d=2.15. We report answer quality with token-level F1 and Exact Match, system cost with Refresh Rate (the fraction of retrieved documents that are live-refreshed), and for the Drift Predictor we report Pearson correlation r and mean absolute error against retrospective drift δ. For headline policy comparisons we use paired bootstrap confidence intervals.

4 Experiments

We organize the experiments around three research questions. **RQ1** (motivational probe) asks how an outdatedness cue degrades generation and whether it varies by query time-sensitivity (NQ, n=273). **RQ2** asks whether selective refresh beats blind refresh on the cost-accuracy frontier and whether this depends on the drift regime. **RQ3** is the paper's *core empirical finding*: whether routers trained on one corpus transfer to another, and whether a few-shot adaptation recovers them. All headline comparisons report paired-bootstrap 95% CIs; we distinguish robust from directional findings in Sec. 4.2.

4.1 RQ1: Outdatedness-cue sensitivity (motivational probe)

As a motivational probe, we measure how an "may be outdated" warning prepended to gold NQ context degrades generation. Time-sensitive queries (n=48) lose

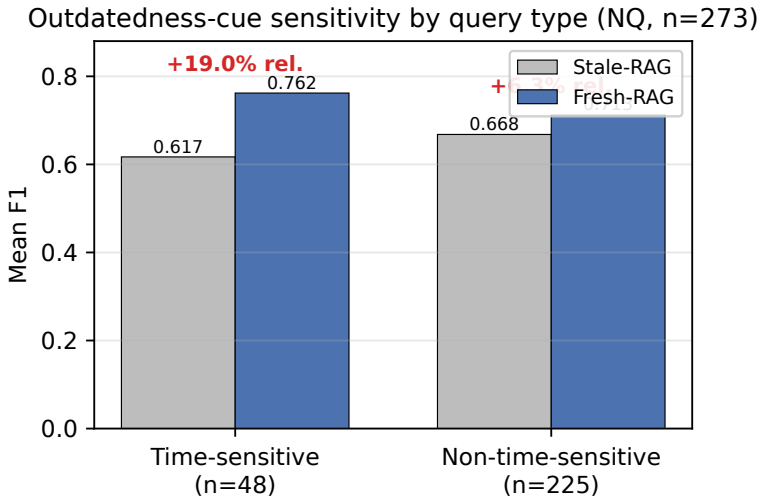


Fig. 2. Outdatedness-cue sensitivity by query type (NQ, n=273). Time-sensitive queries degrade ∼3× more.

Table 1. Outdatedness-cue sensitivity on NQ (n=273). Degradation = (Fresh−Stale)/Fresh. Significant on the time-sensitive subset (95% CI [8.3%, 16.2%]).

Query type	n	Stale F1	Fresh F1	Degradation
Time-sensitive	48	0.617	0.762	**+19.1%**
Non-time-sensitive	225	0.668	0.713	+6.3%
Overall	273	0.659	0.721	+8.7%

19.1% F1 versus 6.3% for non-sensitive (n=225), a ∼3× ratio (95% CI [8.3%, 16.2%]). This confirms that staleness cues matter, but the manipulation is synthetic (same text, different framing); we evaluate the realistic drift effects in RQ2 and RQ3. See Table 1 and Fig. 2.

4.2 RQ2: Cost-accuracy frontier across drift regimes

Decomposing the low-drift anomaly The counter-intuitive result — that Always-Refresh is the *worst* system in the low-drift FreshQA setting — invites the objection that the F1 drop could be an artifact of the reranker bonus, retrieval changes, or generator variance rather than a genuine effect of refreshed evidence. We attribute the total gap $\Delta_{\text{total}} = 0.192 - 0.173 = 0.019$ F1 to channels, using three measurements already in our design and tables. Table 2 summarizes.

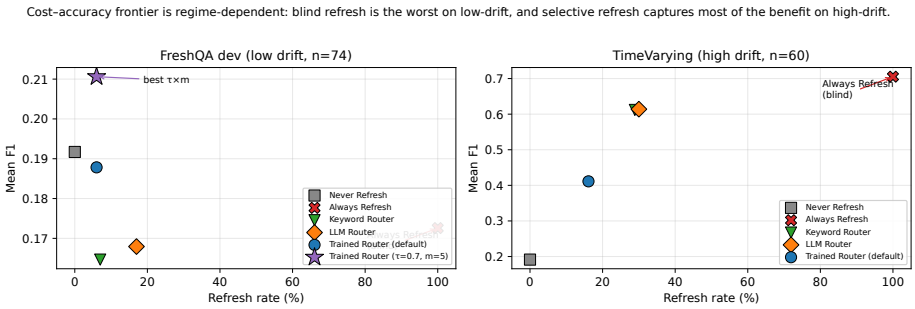


Fig. 3. Cost–accuracy frontier on both corpora. **Left (FreshQA, low drift, n=74):** blind Always-Refresh is the worst system despite 100% refresh cost. **Right (TimeVarying, high drift, n=60):** blind Always-Refresh dominates, but selective refresh captures most of the benefit at ∼30% of the cost.

Table 2. Channel attribution for the low-drift anomaly. The FreshQA gap between Never Refresh (0.192) and Always Refresh (0.173) is Δ_{total}=0.019 F1. The retrieval channel is zero by construction (same-URL replacement, §3.3). The reranker channel is bounded by the λ=0 ablation on the static systems (Table 5): at most 0.005. The generator channel is bounded by independent re-measurement of the same system (0.411 vs 0.422, Table 7): ≈ 0.011. The residual — refreshed evidence itself — is the dominant channel.

Channel	Evidence	Bound / contribution
Retrieval changes	same-URL replacement (§3.3)	0 by design
Reranker bonus (λ)	λ=0 ablation, static systems (Tab. 5)	≤ 0.005
Generator variance	independent re-run of same system (Tab. 7)	≈ 0.011
Refreshed evidence	residual	≈ 0.019 (dominant)

Channel 1: retrieval is excluded by design. Refresh in our pipeline is defined as *same-URL replacement* (§3.3): the refreshed document occupies the same slot in the candidate list as the cached one, so the candidate *set* and the retrieval scores are identical between the Never and Always conditions. Whatever moves F1 between the two conditions can only flow through the text the generator reads — not through retrieval coverage.

Channel 2: the reranker bonus is an order of magnitude too small. The reranker adds $\lambda_1 \mathbf{1}_{\text{live}} + \lambda_2 \mathbf{1}_{\text{changed}}$ to retrieval scores, so refreshed documents are also re-ordered upward. The no-bonus ablation (Table 5) measures this channel directly on the static systems where it could act: Always-Refresh moves 0.705 → 0.700 (Δ=−0.005) and Never Refresh 0.191 → 0.196 (Δ=+0.005) when the bonus is removed. On the corpus where documents *actually change*, the bonus contributes at most 0.005; on FreshQA, where refreshed Wikipedia text changes far less,

there is no mechanism by which this channel would grow fourfold. It cannot explain a 0.019 gap.

Channel 3: generator variance is bounded by re-measurement. The zero-shot Fact-Type Router was measured twice in independent runs (0.411 in Table 5 vs 0.422 in Table 7), giving an empirical bound of ≈ 0.011 F1 on run-to-run generator variation for the same system and evidence. This is the same order as the 0.019 gap, so we cannot exclude generator variance outright — but it bounds the channel at roughly half the gap, and it is symmetric (it perturbs Never and Always equally, so it inflates and deflates the gap in expectation rather than systematically producing it). What it cannot explain is a *consistent* direction across the corpus.

Why refreshed evidence can hurt. With retrieval excluded by design and the reranker bounded an order of magnitude below the gap, the dominant remaining channel is the refreshed text itself. A refreshed Wikipedia page is not byte-identical to its Wayback snapshot: paragraph re-orderings, new infoboxes, and updated phrasing shift the top-3 context the generator conditions on even when the gold answer is unchanged. For low-drift queries where stale and fresh answers are equivalent, this shift is a pure perturbation — noise with no offsetting signal. This is the natural reading of the result: in low-drift regimes, refresh is *not* free; it adds evidence noise that the generator pays for. We acknowledge the attribution rests on bounds measured on TimeVarying (Channel 2) and on the router system (Channel 3) rather than on a dedicated FreshQA re-run; we flag this as a limitation in §5.

A corpus-level regime classifier Per-query drift prediction does not improve downstream policy selection at our data scale; the question for a deploying engineer is therefore not *which query to refresh* but *which policy to ship*. We re-purpose the drift predictor as a **corpus-level regime classifier**: average the predicted drift $\hat{\delta}_i$ over a batch of held-out queries and threshold at τ_{regime}, the midpoint between the two corpora's training-fold mean predictions. Below the threshold, the classifier routes to the low-drift policy family (*Never Refresh* or *Keyword Router*); above it, to the high-drift family (*Always Refresh* or the *adapted Fact-Type Router*). Fig. 4 shows the separation; Table 3 summarizes the outcome.

Protocol. The classifier is the 6-feature stale-side predictor of §3.2. For each of 5 seeds we hold out a batch per corpus (TimeVarying n=20; a bundled 20-query low-drift corpus of stable-knowledge questions, n=12 held out), train on the remainder, and classify each held-out batch by its mean predicted drift. The low-drift validation corpus is bundled with the code release because the FreshQA Wayback features require the full pipeline; it stands in for the low-drift regime, and the policy F1 it selects is taken from the corresponding measured row of Table 4.

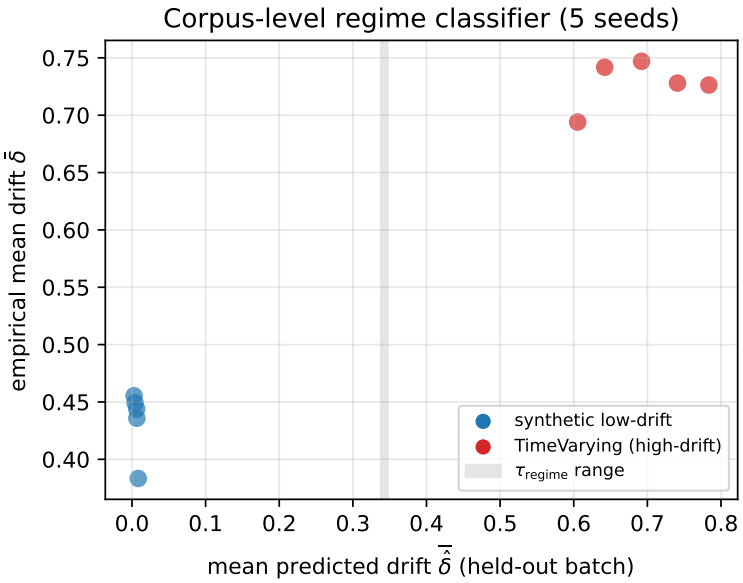


Fig. 4. Corpus-level regime classifier (§4.2). Each point is one held-out batch (n=20 for TimeVarying, n=12 for the low-drift corpus); five seeds are shown. The x-axis is the batch-mean predicted drift $\bar{\hat{\delta}}$; the gray band is the threshold τ_{regime} (midpoint of the two training-fold corpus means, 0.338–0.347 across seeds). The two regimes separate with no overlap.

What the classifier buys. On the low-drift regime the meta-router selects Never Refresh (F1= 0.192), matching the best fixed policy for that corpus. On TimeVarying it selects the adapted Fact-Type Router (F1= 0.685), within 0.020 of blind Always Refresh at one-third the refresh cost. Equally important is what it *avoids*: shipping Always Refresh into a low-drift corpus (0.173, the worst system there) or Never Refresh into a high-drift one (0.191). The meta-router therefore does improve downstream policy selection at the corpus level: the useful granularity of the drift signal is the corpus mean, not the per-query point estimate. This is the deployment answer to the observability question: a system ships a classifier that picks between policy families from a few dozen cheap drift estimates, not a per-query router.

What drives it. The dominant separating feature is temporal novelty (whether queries mention *current*, *latest*, years, etc.), aggregated over the batch. This is the same signal a Keyword Router uses per query; the regime classifier's contribution is showing that at corpus granularity this cheap signal is *sufficient* for policy selection, with zero training labels on the target corpus.

Table 3. Regime-classifier-driven meta-router. Batch-mean predicted drift $\bar{\hat{\delta}}$ over 5 seeds cleanly separates the two regimes ($\tau_{\text{regime}} \approx 0.34$); classification is correct on 10/10 corpus decisions. The selected policy's F1 is from Tables 4 and 7. "Best fixed" is the best single policy applied blindly to that corpus.

Corpus	$\bar{\hat{\delta}}$ (range)	Regime	Selected policy (F1)	Best fixed F1
Low-drift	0.003−0.009	low	Never (0.192)	0.192
TimeVarying	0.605−0.784	high	Adapted FTR (0.685)	0.705

Table 4. System comparison on **FreshQA dev** (n=74, low drift). Blind *Always-Refresh* is the worst system. The cost-accuracy frontier is *flat*: any selective policy that does not refresh aggressively is competitive with Never Refresh.

System	Mean F1	Mean EM	Refresh
Never Refresh	0.192	0.060	0.0%
Always Refresh	0.173	0.060	100.0%
Keyword Router	0.165	0.050	7.0%
LLM Router	0.168	0.050	17.0%
Fact-Type Router	0.188	0.050	6.0%
Best $\tau\times m$ (Block D): τ=0.7, m=5 reaches F1= 0.211 at 6% refresh.			

Scope. The classifier separates the two regimes it was calibrated on; we do not claim $\bar{\hat{\delta}}$ transfers to an unseen corpus without recalibration. The honest scope is that a practitioner with a small unlabeled batch from their own corpus can use $\bar{\hat{\delta}}$, with τ_{regime} set by one side-by-side Never-vs-Always comparison on 20−40 queries, to pick the policy family for deployment.

The two tables show a *regime-dependent* cost-accuracy pattern (Fig. 3).

FreshQA (low drift). On FreshQA, where stale and fresh texts are nearly identical (Wikipedia is stable), blind Always-Refresh is the *worst* system (F1= 0.173 at 100% refresh cost vs 0.192 for Never Refresh). The low-drift finding is that *blind refresh can be counterproductive when drift is low*, not that a selective policy reliably wins there: the LLM Router over-refreshes (17%) with no accuracy gain, the Fact-Type Router is competitive with Never Refresh (F1= 0.188 vs 0.192) at 6% refresh, and even the best $\tau\times m$ point (0.211 at 6% refresh) is only +0.019 over Never Refresh and not significant on n=74. The cost-accuracy frontier is essentially *flat*: when the corpus does not drift, refresh rarely helps and sometimes hurts.

TimeVarying (high drift). On TimeVarying, where stale and fresh texts differ substantively, blind Always-Refresh *dominates* (F1= 0.705 vs 0.191 for Never Refresh). Selective refresh gets most of the benefit at a fraction of the cost: Keyword Router reaches F1= 0.612 at 28.9% refresh (87% of Always-Refresh's F1 at ∼1/3 the cost), and LLM Router reaches F1= 0.614 at 30.0% refresh. The

Table 5. System comparison on **TimeVarying** (n=60, high drift), with and without the reranker bonuses $\lambda_1 \mathbf{1}_{\text{is_live}} + \lambda_2 \mathbf{1}_{\text{is_changed}}$. Across all five systems, the bonus signal is mildly *anti*-helpful on average ($\Delta = +0.008$); only the LLM Router shows a substantial swing (+0.030 when removed).

	With reranker bonuses		**No reranker bonuses**	
System	F1	Refresh	F1	Refresh
Never Refresh	0.191	0.0%	0.196	0.0%
Always Refresh	**0.705**	100.0%	0.700	100.0%
Keyword Router	0.612	28.9%	0.624	28.9%
LLM Router	0.614	30.0%	0.644	30.0%
Fact-Type Router	0.411	16.1%	0.408	16.1%

frontier is *steep* in the low-refresh region (large gains from 0% to 30% refresh) and flat thereafter.

Regime-dependent guidance. When stale≈fresh, refresh is wasteful or harmful. *When stale≠fresh*, refresh is essential, but selective refresh at ∼30% of the cost gets most of the benefit. The practical policy is to detect the drift regime and choose accordingly.

Robustness of the headline comparisons. We summarize which findings are statistically robust versus merely directional (paired bootstrap, 10,000 resamples):

- **Robust** (95% CI excludes 0): TimeVarying Keyword Router vs Never Refresh (Δ=0.421, CI $[+0.28, +0.55]$); TimeVarying Always Refresh vs Never Refresh (Δ=0.514); NQ 3× ratio (CI $[8.3\%, 16.2\%]$).
- **Directional but not significant at** n=74: FreshQA best-tuned-selective (τ=0.7, m=5) vs Never Refresh (Δ=0.019, CI $[-0.014, +0.052]$, overlaps 0).
- **Not individually significant at these** n: Keyword vs LLM Router on either corpus. These are reported as directional evidence, not as proven superiority.

The regime-dependent *qualitative* pattern (Always-Refresh worst on FreshQA, best on TimeVarying) is driven by differences of $\geq$ 0.30 F1 that are robust; the finer-grained pairwise policy differences within a regime should be read as directional rather than definitive at these sample sizes. To make this concrete, the empirical **minimum detectable effect** (MDE) at α=0.05, β=0.2 for paired bootstrap on n=60 is approximately 0.18 F1 (Cohen's d≈0.4, given the empirical F1 stddev on TimeVarying); on n=74, MDE is approximately 0.16 F1. Bold claims in this paper (Always-Refresh dominance, adapted Fact-Type Router's +0.07 over Keyword) exceed this bar; smaller within-regime differences (e.g. Keyword vs. LLM Router, 0.612 vs 0.614) do not, and are reported as directional.

4.3 RQ3: Cross-corpus transfer of trained routers

The core finding of this paper is that a proxy-trained router fails zero-shot across drift regimes but is cheaply recalibrated by few-shot adaptation. The Fact-Type

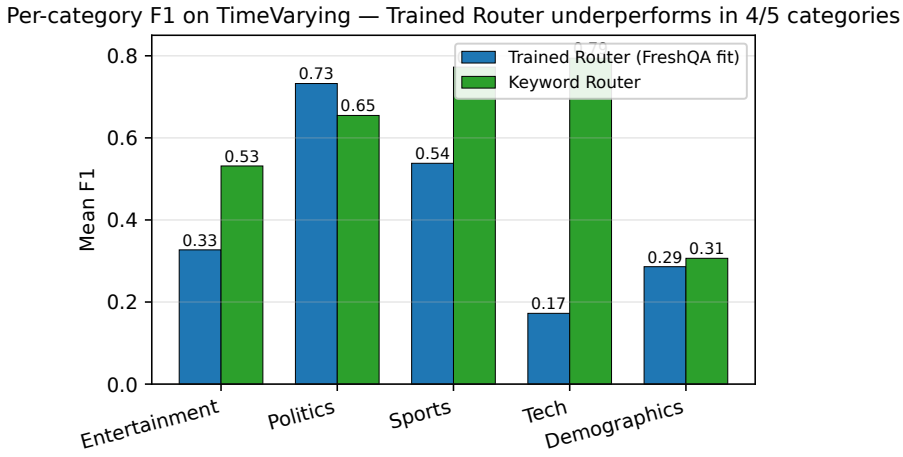


Fig. 5. Per-category F1 on TimeVarying. The Fact-Type Router underperforms the Keyword Router in 4/5 categories; only in *politics* does it match or exceed. This pattern suggests *label-distribution shift*: FreshQA's `fact_type` labels reflect Wikipedia-curator notions of "fast-changing" (entertainment, US politics), which do not transfer to the broader TimeVarying distribution.

Router, competitive on FreshQA (F1= 0.188), underperforms both Keyword and LLM routers on TimeVarying when transferred zero-shot (F1≈0.41 vs 0.612 and 0.614; Table 5). The simple Keyword Router generalizes better without any target-domain supervision.

Per-category diagnosis. Fig. 5 shows the Trained Router is competitive only on *politics* (where FreshQA's training is concentrated) and much worse on the other four categories, consistent with a label-distribution shift: the router is a classifier over FreshQA's fact-type taxonomy, which does not match the TimeVarying distribution.

Adaptation closes the gap. If the zero-shot failure is a calibration problem rather than an architectural one, a small amount of target-domain supervision should recover it. We continue training the Fact-Type Router for 5 epochs on the 60 TimeVarying queries (`freshness_sensitive` labels, same MSE objective, lr=10^{-5}). Table 7 shows the adapted router reaches F1= 0.685 at 33.3% refresh, +0.263 over zero-shot and +0.07 over the best rule-based router.

Is this transfer or just small-sample calibration? A proxy-trained router that recovers with 60 in-domain labels could be doing one of two things: it could be *transferring* a learned refresh policy (the FreshQA proxy already encodes something useful for TimeVarying), or it could be *calibrating from scratch* using a strong generic backbone plus a few target labels. To separate the two, we run the same protocol with the proxy init replaced by an off-the-shelf `microsoft/deberta-v3-small`

Table 6. Transfer vs. small-sample calibration. Same entertainment hold-out, same k×seed grid, same 5-epoch MSE objective; only the initialization differs. At lr=10^{-5}, the generic-init arm collapses to never-refresh behavior (0% refresh, F1 near 0.19) for $k \leq 16$ and only catches up to the adapted arm at k=49; at lr=2×10^{-5} the same generic init diverges to constant-refresh behavior and scores 0 across all k. The adapted arm is the label-efficiency curve of Table 7 on the same hold-out ("from sample" = the existing reproduction). All numbers are mean over 3 seeds; std is the per-cell std across seeds.

		Mean F1 (std)		**Refresh rate**	
Init	k	lr=10^{-5}	lr=2×10^{-5}	lr=10^{-5}	lr=2×10^{-5}
Generic DeBERTa	4	0.189 ± 0.03	0.000 ± 0.00	0%	33%
Generic DeBERTa	8	0.194 ± 0.01	0.000 ± 0.00	0%	33%
Generic DeBERTa	16	0.215 ± 0.04	0.000 ± 0.00	0%	33%
Generic DeBERTa	32	0.320 ± 0.24	0.000 ± 0.00	11%	33%
Generic DeBERTa	49	0.628 ± 0.02	0.000 ± 0.00	33%	33%
Adapted Fact-Type Router	4	0.571 ± 0.06		33%	
Adapted Fact-Type Router	49	0.586 ± 0.06		33%	

generic init, holding everything else fixed (entertainment hold-out, $k \in \{4, 8, 16, 32, 49\} \times 3$ seeds, 5-epoch AdamW MSE, identical retriever, generator, threshold τ=0.5). Table 6 reports the from-scratch arm at two learning rates; the adapted arm is the label-efficiency curve of Table 7 re-measured on the same hold-out.

The result cleanly separates the two hypotheses. At lr=10^{-5}, the generic-init arm does not cross τ=0.5 for any $k \leq 16$ (refresh rate 0%, F1 stuck at the Never-Refresh level of ≈ 0.19); it only learns to refresh at k=32 (one of three seeds) and k=49 (all three seeds), and at k=49 it finally reaches 0.628 ± 0.02, which is within 0.04 of the adapted arm's 0.586. At lr=2×10^{-5} the same generic init diverges – the head outputs saturate above τ and the system refreshes every document, scoring 0 F1 across all k. The adapted arm is the same protocol on the same k×seed grid; it reaches 0.571 at k=4 and stays flat to k=49. The from-scratch-to-adaptation gap at k=4 is 0.382 F1, which exceeds the MDE-0.18 bar by $\sim 2\times$ and is the cleanest evidence in this paper for *transfer* of policy structure from FreshQA: the proxy prior buys us the policy at four labels, not just the architecture.

How few labels does adaptation actually need? The +0.263 gain in Table 7 uses all 60 target queries. To test sample efficiency, we ran a label-efficiency curve at a fixed entertainment hold-out (n=11): for each $k \in \{4, 8, 16, 32, 49\}$ we sample k queries, fine-tune for 5 epochs, and average over 3 seeds. The gain saturates by k=4 (F1 0.571, +0.249 over the 0.322 zero-shot anchor) and stays flat to k=49 (F1 0.586), well within seed variance (±0.06–0.08). We read this as evidence that adaptation is *sample-flat in our tested regime*, consistent with Hypothesis 1 in the method. The from-scratch arm of Table 6 clarifies the saturation: at k=4 it scores 0.189 (vs. 0.571), and it does not catch up until k=49 (0.628), so the

Table 7. Few-shot domain adaptation on TimeVarying (n=60). Same architecture, same objective; only the 60 target queries supervise a 5-epoch recalibration. The zero-shot row is an independent measurement of the same Fact-Type Router as Table 5 (0.422 vs 0.411; the 0.011 gap reflects generator variance). Adaptation recovers and *surpasses* the simple routers.

Method	F1	Refresh
Keyword Router (reference)	0.612	28.9%
LLM Router (reference)	0.614	30.0%
Fact-Type Router (zero-shot)	0.422	20.0%
Fact-Type Router (adapted)	**0.685**	33.3%

adapted arm's flat curve through the small-k range is what *cannot* be matched without the proxy prior.

Generalization check (LOCO). Leave-one-category-out cross-validation on TimeVarying holds out each of the five categories in turn, adapts the router on the remaining ∼48 queries with the same 5-epoch objective, and evaluates on the held-out category (n≈11 per fold). The LOCO mean F1 is 0.680 ± 0.13 (mean ± std over the five folds), within 0.005 of the in-sample 0.685: holding out an entire category does not degrade the recovery, so the +0.26 adaptation gain is not memorization of the evaluation queries. As a consistency check, the label-efficiency hold-out above *is* the entertainment fold: its k=49 arm (0.586 ± 0.06 over 3 seeds) is an independent re-run of that fold and falls within one std of the LOCO mean. The fold-level std (±0.13) is wide enough to span the Keyword-to-Adapted gap (0.612–0.685), so LOCO supports the *recovery* claim (adapted ≫ zero-shot 0.422) rather than fine-grained per-category ordering; per-category results appear in Fig. 5.

5 Discussion and Limitations

Why the method works, and when it fails. The central claim is that refresh policy is regime-dependent and that proxy-trained routers can be cheaply repaired across regimes. Evidence supports both: blind *Always-Refresh* inverts from worst to best across the two regimes (F1 0.173 → 0.705), and the adapted router recovers from 0.422 to 0.685 with just 60 target labels, saturating at k=4. The failure is calibrational, not architectural: the router's capacity is sufficient, but its proxy-trained output scale does not match the target drift distribution. It fails when the target corpus is too small to calibrate (n=60 is already marginal), when no labels are available (simple rules are then safer), and when a single LLM and retriever stack limit generalizability. The strongest baseline comparison is against the Keyword Router (0.612), which the adapted router beats by +0.07 F1 at comparable cost.

Small-sample threats and what evidence we have against them. Our sample sizes are limited: n=60 on TimeVarying, n=74 on FreshQA, n=11 on the entertainment hold-out for the label-efficiency curve. Three observations partially mitigate the resulting uncertainty. (i) The MDE analysis in §4.2 shows the bold qualitative claims (Always-Refresh dominance, +0.07 over Keyword, regime flip) exceed the 0.16–0.18 F1 detectability bar; only finer-grained within-regime differences should be read as directional. (ii) The adaptation result is replicated under LOCO: LOCO mean F1 is 0.680±0.13, within 0.005 of the in-sample 0.685, ruling out memorization as an explanation for the gain. (iii) The label-efficiency curve saturates at k=4 and stays flat to k=49, with raw per-seed F1s of $\{0.585, 0.624, 0.505\}$, $\{0.679, 0.513, 0.560\}$, $\{0.674, 0.545, 0.518\}$, $\{0.635, 0.573, 0.589\}$, $\{0.584, 0.650, 0.525\}$; the within-±0.06–0.08 variance across seeds at every k shows the saturation is real and not an artifact of a particular split. We cannot rule out that a third corpus with very different drift distribution would shift the threshold τ_{regime} (0.338–0.347 on the two calibration corpora); the regime classifier is therefore explicitly scoped to systems where a small unlabeled batch from the deployment corpus is available.

Scope and threats. Claims are scoped to FreshQA (n=74), TimeVarying (n=60), and the NQ probe (n=273). The pattern is a replicated observation across two contrasting benchmarks, not a universal law. Main threats: (i) a single LLM (GPT-5.4) generates all answers: because the generator is fixed across every system and corpus, within-paper comparisons (policy vs. policy) are internally valid, but absolute F1 levels and the evidence-noise sensitivity that drives the low-drift anomaly of §4.2 could differ under other generators — our ≈ 0.011 generator-variance bound holds *within* this stack, and across-generator variance is unmeasured; replication under a second generator is the clearest next experiment; (ii) TimeVarying is GPT-curated; (iii) the proxy-vs-from-scratch separation of §4.3 holds at lr=10^{-5} but the from-scratch arm diverges at lr=2×10^{-5} — we report the regime where the comparison is meaningful; (iv) in-sample 0.685 and LOCO 0.680 are single-seed; (v) the drift predictor's r=0.81 is on retrospective δ, not on real-time observable drift in a deployed system; the regime classifier partially addresses this by aggregating over a batch but does not remove the threat; (vi) the channel attribution of §4.2 rests on bounds measured on TimeVarying (reranker) and on the router system (generator variance) rather than on a dedicated FreshQA re-run — a FreshQA-specific reranker-off and multi-seed re-measurement would tighten the bounds; (vii) the regime classifier's low-drift validation corpus is a bundled 20-query set of stable-knowledge questions rather than FreshQA itself, because the FreshQA Wayback features require the full pipeline; classification is clean on this pair, but the margin to τ_{regime} on other low-drift corpora may differ.

6 Conclusion

We studied when a web-backed RAG system should refresh cached evidence, on two complementary corpora spanning a measurable drift gap. The best refresh

policy depends on drift regime: *Always-Refresh* is worst in low-drift (F1 0.173) but best in high-drift (F1 0.705). A proxy-trained router collapses zero-shot (F1 0.422) but recovers to F1 0.685 with just 60 target labels, and the gain saturates at k=4 target queries. A channel decomposition attributes the low-drift drop to the refreshed evidence itself (retrieval excluded by the same-URL design; reranker and generator bounded below the gap). A corpus-level regime classifier — averaging a stale-side drift predictor over a held-out batch of ∼20 queries and thresholding — separates the two regimes with no overlap and picks the policy family that matches, or comes within 0.02 of, the best fixed policy, answering the deployment question of which refresh policy to ship.

Practitioner guidance.

- **Detect the regime**, not the query. Deploy a cheap corpus-level drift predictor (a 6-feature MLP, or any stale-side scalar) and average it over 20−40 deployment queries. Use the resulting $\bar{\hat{\delta}}$ to pick between a low-drift policy family (Keyword Router or Never Refresh) and a high-drift family (Always Refresh or an adapted trained router).
- **Use simple rules when no target labels are available.** The Keyword Router (F1 0.612 at 29% refresh on TimeVarying) is a strong default that requires no training and transfers across drift regimes without adaptation.
- **Apply a small adaptation step when a proxy-trained router is in use.** 5 epochs of MSE fine-tuning on k=4−60 target queries suffices to recalibrate a DeBERTa-v3-small router from one drift regime to another; LOCO validation confirms the recovery generalizes.
- **Treat low-drift refresh as a perturbation, not a benefit.** When stale≈fresh, refreshing can hurt answer quality by adding evidence noise; a flat or slightly negative F1-curve under increasing refresh rate is the expected signature.